CS 30700: Sprint 1 Retrospective

BoilerSpace

Team 3: Aagam Bapna, Sheehan Dandapat, Rohan Potta, Nathan Wan, Jed Macapinlac, Bidit Acharyya

What went well?

- **User Story 51: As a student, I want to view and upload notes from study sessions for my course so that I can both review material I may have missed and contribute to the shared knowledge base.** This user story is an integral part of our app as it covers one of the main features in note uploading. This story was difficult to implement as we had to save the files uploaded by users into our MongoDB database as Buffer variables so that other users can download them as well. We were able to successfully do this, which helped bring us a lot closer to the finished product.
- **User Story 40: As a student looking for a seat, I want to view an interactive campus map so that I can quickly see buildings and rooms.** This user story is also central to our app as the map is the primary interface that users will interact with. We used Mapbox to get the map of campus, and we added color coded markers to academic buildings so users can easily identify study spots and check availability.
- **User Story 31: As a student, I want to see a basic list of buildings sorted alphabetically so I can find my favorite study spot quickly.** This user story was a major part of our application as it acts as the gateway to building selection, enabling users to access one of our core features: study spot discovery. Using data from Purdue's official building directory, a small list of academic buildings was successfully uploaded and integrated into our database. As development continues, we expect to further expand the building database.
- **User Story 30: As a student, I want to click a single button to check into a room so I can contribute to live data without filling out a long form.** This user story directly improved the usability of our application. With the ability to efficiently check into rooms and buildings, users are better enabled to contribute

to our crowdsourced data, which improves the overall functionality and accuracy **of the app.**

- **User Story 32: As a student, I want to see how many people are currently checked into a specific room or building so I can gauge crowd levels.** This user story was successfully implemented, allowing users to view real-time occupancy counts for rooms and buildings. By displaying live check-in data, this feature gives students a quick and easy way to assess how busy a space is before heading there, directly supporting the app's goal of helping users find the best available study spots.
- **User Story 33: As a student, I want to use a search bar to find a specific building by its name or campus abbreviation.** This user story was integral for the UI as it enables for the student to find their favorite buildings on campus at the tip of their fingertips. By being able to provide search for all buildings in the purdue database as well as the abbreviations that they go by, we gave students ease of use as if they have been using this service forever.
- **User Story 40: As a student looking for a seat, I want to view an interactive campus map so that I can quickly see buildings and rooms.** This was an integral part of the project, as it was the main landing page UI, and it's a great relief that this has gone as planned. By sending the user straight to Purdue's campus on the map, it gives the user an optimal view from which they have a bird's eye view of what's available on campus to study at.
- **User Story 43: As a student, I want to bookmark my favorite rooms so that I can quickly check them in the future.** This was one of the features for the project that we felt was necessary for the user to truly feel that this is a useful app. Everyone has their go-to spots on campus, and having this integrated into our project was integral to serve our user base.
- **User Story 1: As a user, I would like to register for a BoilerSpace account so that I can save my preferences and access the full suite of features.** This user story was a critical foundation for the app since every other feature depends on users being able to create an account. Getting the MongoDB schema right early on made it easy to build on top of for login and verification. Bcrypt password

hashing was straightforward to set up and gave us confidence that user data is being stored securely. Connecting the React form to the backend also went smoothly and gave us a fully working registration flow by the end of the sprint.

- **User Story 2: As a user, I would like to be able to login and manage my BoilerSpace account so that I can keep my personal data secure and up to date.** This user story was essential because without authentication, none of the protected features in the app would be accessible. Passport.js and JWT worked well together and made it easy to secure endpoints without a lot of extra code. Having protected route middleware in place early means the rest of the team can lock down new routes as they build them out. Overall this gave the app a solid and scalable auth system going forward.
- **User Story 3: As a user, I would like to be able to verify my Purdue email by getting an OTP so that the platform remains exclusive to the campus community and prevents spam.** This user story was important for keeping BoilerSpace a trusted and campus only platform. Restricting registration to @purdue.edu emails ensures that only real Purdue students can access the app. The 15 minute OTP expiration adds a layer of security and prevents old codes from being reused. Getting the email service integrated and working reliably was one of the more interesting challenges of the sprint and ended up coming together well.
- **User Story 5: As a user, I would like to be able to edit my profile (display name, major, year) so that others recognize me in sessions.** This user story was important as it allowed users to edit their profiles to reflect any changes in their academic status, such as their graduating year or major. This was successful as user changes are successfully saved.
- **User Story 52: As a student, I want to upvote/downvote certain notes so that the most helpful and accurate resources rise to the top.** This user story is important as it is a core feature of our app. It allows users to place votes on notes, which lets other users know about the quality of the posted notes and know which notes to cover for effective study sessions. We were able to get the votes to persist when the site was reloaded, ensuring all votes are accurately recorded.
- **User Story 8: As a user I want to set my classes for this semester, so that others know what I am studying.** This user story is important as it allows users

to set their course lists so that they can see all notes associated with their courses and also see what courses other users use. The course lists are pulled from the MongoDB database and the selections are saved so that the list is preserved between sessions and reloads.

- **User Story 35: As a student, I want to see a list of valid Purdue courses (e.g., CS 30700) so that I can select the correct ones for my profile and finding study partners.** This user story is important because it is an integral part for users to be able to select which classes they are in so they can view important information about the class and view notes uploaded by other users for that specific class. We were able to get the catalog set up with many of the integral classes for this sprint, and users could also see the course notes for their specific class.
- **User Story 4: As a user, I would like my password to be reset if I forget it so that I can regain access to my account without creating a new one.** This user story is important because it is a common part of user registration and login features, as many users tend to forget their password. We were able to get this working by generating a specific token for users specific to their email when they request to change their password. They can access the token once they receive the reset password email which goes directly to their Purdue email. From there the link validates the user with the token and allows them to reset the password as the UI is set up for it. From there they can login in successfully.
- **User Story 71: As a club organizer, I want to create and manage a club profile so that students can learn about the organization.** This user story is important because it allows student organizations to establish a presence within the app and share information about their club with other students. It enables organizers to provide key details such as the club name, description, contact information, and category so that users can easily discover and learn more about different organizations. We were able to successfully implement the club creation and retrieval functionality for this sprint, and the authorization system ensured that only the club organizer could edit or update the club profile. Organizers were also able to modify their club information and see those updates reflected immediately in the application.
- **User Story 62: As a club organizer, I want to post events with time and location so that students can discover them easily.** This user story is important because it allows clubs to promote their activities and helps students find events that they may want to attend. By providing clear event information such as the title, date, time, location, and associated club, users can easily browse upcoming events and stay informed about what is happening on campus. We were able to successfully implement the event creation and retrieval functionality for this sprint, and the system ensured that only authorized club

organizers could create events for their club. This allowed events to be displayed properly and organized for users to view.

What did not go well?

Overall, Sprint 1 was a strong start for the team. We successfully delivered a large number of core features and the team showed great collaboration and technical ability by getting so many moving parts working together in a single sprint. That said, we ran into some challenges toward the end of the sprint. Last-minute integration caused some stress, merge conflicts piled up as everyone pushed to main near the deadline, and some features were not fully tested end-to-end before the demo. We also noticed that communication around task dependencies could have been clearer, as some teammates were unknowingly blocked waiting on others. Overall we are proud of what we shipped in Sprint 1 and are confident that the improvements we have planned will make Sprint 2 even smoother.

Specific Stories with Issues:

- **User Story 68: Event Announcements & Updates:** During the live demo, the announcement posting flow encountered an issue where announcements could not be successfully created from the event page due to an error in the request path and authorization validation. This prevented the announcement from appearing on the event page during the demo. We were able to quickly identify the issue and the bug was quickly fixed, so the feature now functions correctly. However, this situation showed us that the full end-to-end flow from the frontend to the backend had not been tested thoroughly in the exact scenario used during the demo. Moving forward, we plan to do more complete integration testing before demonstrations to ensure that all user-facing workflows function as expected.

How should we improve?

1. Define and enforce acceptance criteria earlier in the sprint, written specifically around live demo scenarios. In Sprint 1, several of our acceptance criteria were written from a technical/backend perspective (e.g., "the system prevents duplicate check-ins") rather than from the user's point of view as they'd experience it in a live demo. Going forward, we should write every acceptance criterion as a concrete, demonstrable user action before any development begins, something like "Given I check in to WALC 1087, when I try to check in again, then I see a message saying I'm already checked in." This makes it crystal clear what "done" looks like for both the developer and the person demoing, and avoids last minute scrambles to make features presentable.

2. Adopt a more disciplined Git merging workflow to avoid bottlenecks on main. We used feature branches this sprint, which was a good start, but during high volume periods especially toward the end of the sprint main got bombarded with simultaneous pull requests and pushes. This led to occasional merge conflicts, broken builds, and teammates having to rebase or pull constantly, which slowed everyone down. Next sprint, we want to spread merges more evenly throughout the sprint instead of saving them all for the last few days. We'll also require at least one teammate to review a PR before it gets merged so that conflicts and issues get caught early rather than piling up.

3. Improve communication around task dependencies across team members. Some of our user stories had hidden dependencies, for example, Bidit's course selection UI (Story 8) relied on Rohan's course catalog API (Story 35) being functional first. When those dependencies weren't called out early, one person would be blocked waiting on another without the team realizing it. Next sprint, during planning we'll explicitly map out which tasks depend on which, flag them in our Monday meetings, and make sure the blocking task gets prioritized so no one is sitting idle mid-sprint.

4. Establish a code freeze before the demo and build stronger cross-team knowledge of each other's work. This sprint we were pushing backend changes right

up until demo time, which left little room for frontend polish or catching last minute bugs. Next sprint, we want to set a hard cutoff, roughly two days before the demo, where no major backend pushes happen and the team shifts to connecting everything, cleaning up the UI, and rehearsing the demo flow. On top of that, we realized that most of us only deeply understood our own individual pieces but not how our teammates' parts worked. This made integration clunky because we'd run into unexpected API shapes or misunderstood data flows. Going forward, we plan to use our Monday/Wednesday standups to have each person briefly walk through what their endpoints expect and return, so everyone has a working mental model of the full system, not just their slice of it. That shared understanding will make integration smoother and improve the overall coherence of the project.

5. **Keep environment setup documentation up to date as the project grows in complexity**. We did a solid job this sprint getting everyone set up initially, but as the project continues and more complex features get added, new npm packages, additional environment variables, third-party API keys, updated seed scripts, those setup steps will drift from reality quickly. Rather than teammates having to ask each other on Slack how to get something running locally, we want to maintain a shared living document that covers everything needed to run the project from scratch: npm installs, .env file structure with required variables, database seeding commands, and any third party service configuration. Whenever someone introduces a new dependency or config change, updating that doc should be part of the PR itself. This way no one is blocked waiting on someone else to explain how to get their feature running locally for testing or integration.

6. **Improve code-level documentation during active development to support smoother cross-team integration.** Our planning documents such as sprint planning, team charter, backlog, and design docs were thorough and well-formatted, which gave us a strong foundation going into the sprint. However, once active development started, documentation dropped off slightly. When teammates needed to understand how someone else's API worked or what shape a response returned, they were mostly reading through the raw code and figuring it out themselves rather than referencing any

written documentation. For features that heavily depend on each other such as course selection relying on the course catalog API, or the map component needing building retrieval endpoints, this slowed things down and introduced misunderstandings. Next sprint, we want to make lightweight code documentation a standard part of development: brief comments on non-obvious logic, a short description at the top of each route file explaining what it does and what it expects, and keeping a running API reference that lists each endpoint's request format and response shape. This doesn't need to be excessive, even a few lines per file would save teammates significant time when integrating across features.